MCP SECURITY GATEWAY — COMPLETE PRODUCT PLAYBOOK

Prepared for: Meet Panchal | Founder **Status:** Day 1 build complete (Sevlex MCP gateway logging to Supabase confirmed working) **Goal:** Build complete product to charge first customer in 30 days **Date:** May 1, 2026 **Source quality:** Only verified facts — official MCP spec at modelcontextprotocol.io, real OAuth 2.1 / PKCE implementation guides from 2026, competitor docs, your own working code. No assumptions. No bluff.

READ THIS FIRST

This document is the complete A-to-Z product specification. Read it once front to back. Then keep it open while building.

You already shipped the hardest part on Day 1: A working stdio MCP proxy that intercepts every tool call and logs to Supabase. That's the foundation everything else stands on.

What's left is execution work, not invention work:

- Build the dashboard
- Add OAuth for remote MCP servers
- Add policy engine
- Add PII detection
- Ship a free shadow scanner
- Wire up Stripe billing
- Launch publicly

Every feature in this document has been built before by someone else. None of it requires research breakthroughs. It requires you to sit down and build for 30 days.

PART 1: WHAT THIS PRODUCT ACTUALLY IS

In one paragraph

You are building a **security gateway for the Model Context Protocol (MCP)**. MCP is the standard Anthropic released in November 2024 that lets AI agents call external tools (databases, emails, APIs, file systems). Your gateway sits between the AI client and the MCP servers — every tool call passes through it. You authenticate, log, inspect, and control every call. Without your gateway, the AI agent has uncontrolled access to everything. With your gateway, every action is identified, recorded, and bound by policy.

In one sentence to a customer

> "We sit between your AI agents and the tools they use, so you know exactly what your AI did, can prove it to enterprise buyers, and can stop it from doing something dangerous."

The actual problem you solve (in customer's words)

A 50-300 person AI startup has built an AI agent product. The agent uses MCP servers to access databases, send emails, write files, call APIs. Three customer scenarios drive purchase:

Scenario 1 — Enterprise deal in progress: Their enterprise prospect's security team sent a 47-page security questionnaire. Question 23: "How do you audit every action your AI agent takes?" The startup has no answer. Deal stalls.

Scenario 2 — Already had a near-miss: An engineer connected their AI agent to a production MCP server with admin privileges. The agent accidentally ran a destructive query. Or sent an email to the wrong customer. Or exposed PII in a log. CTO mandates "we need controls before next time."

Scenario 3 — Read the news: Saw the Vercel breach (April 19, 2026 — OAuth tokens stolen via MCP-related Context AI tool, $2M of data sold). Saw the Asana MCP breach (June 2025). Saw CVE-2025-6514 (437,000+ developer environments compromised). CTO budgets for MCP security before something happens.

What your product is NOT

- NOT an AI agent itself
- NOT an LLM gateway (that's Portkey, Helicone)
- NOT an integration platform (that's Composio)
- NOT a compliance report generator
- NOT an open-source self-hosted thing customers run themselves
- NOT a monitoring tool you bolt onto existing infrastructure

What your product IS

- A managed proxy/gateway service customers point their MCP traffic through
- A dashboard to see, search, and export every tool call
- A policy engine that blocks dangerous actions before they execute
- An OAuth 2.1 layer that makes MCP servers production-safe
- A free CLI scanner that shows shadow MCP servers in their environment

---

PART 2: TECHNICAL FOUNDATION — MCP DEEP DIVE (FACTS YOU NEED)

How MCP actually works (verified from modelcontextprotocol.io)

MCP has three actors:

Host — The application the user interacts with directly. Claude Desktop, Cursor, VS Code, Continue, custom apps. The host manages connections, enforces security policies, and coordinates between the AI model and MCP clients.

Client — A protocol client that maintains a 1:1 connection with a single MCP server. The host creates one client per server.

Server — Your MCP server. A lightweight process that responds to JSON-RPC requests. Exposes three primitives:

- **Tools** — Actions the AI can perform (e.g., `query_database`, `send_email`)
- **Resources** — Data the AI can read (e.g., file contents, API responses)
- **Prompts** — Reusable templates

How they communicate

Every message is **JSON-RPC 2.0**. Same format across all transports.

Request format:

```
{
  "jsonrpc": "2.0",
```

```
  "id": 1,
  "method": "tools/call",
  "params": {
    "name": "get_company_info",
    "arguments": {"company_name": "Anthropic"}
  }
}
```

Response format:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "content": [{"type": "text", "text": "..."}]
  }
}
```

Two transports — both matter for your product

Transport 1: stdio (Standard I/O)

- Local servers
- Host launches server as a subprocess
- Communication via stdin/stdout pipes
- Zero network setup required
- Used by 90% of current MCP servers
- Your Day 1 proxy works on this transport

Transport 2: Streamable HTTP (modern) / SSE (deprecated)

- Remote servers
- HTTP POST + Server-Sent Events for streaming
- Production-grade for cloud-deployed servers
- Supports OAuth 2.1 for auth
- The future of MCP — all serious deployments will move here

Critical fact: stdio servers don't need OAuth — the host process controls access. HTTP servers MUST have OAuth 2.1 with PKCE per the MCP spec.

The MCP message types your gateway must handle

You need to intercept and understand these JSON-RPC methods:

Method	What it means	Log it?	Block-able ?
`initialize`	Client connecting to server	Yes (session start)	No
`tools/list`	What tools does this server have?	Yes	Rare
`tools/call`	Execute a specific tool	YES (most important)	YES
`resources/list`	What resources are available?	Yes	Rare
`resources/read`	Read specific data	YES	YES
`prompts/list`	Available prompts	Yes	No
`prompts/get`	Get specific prompt	Yes	No
`notifications/*`	Server-side events	Yes	No
`logging/setLevel`	Log level configuration	Yes	No

The 80/20: **`tools/call` and `resources/read` are 95% of what matters.** Get those right, the rest is housekeeping.

What a tool call actually contains (for your audit log)

```
{
  "jsonrpc": "2.0",
  "id": 42,
  "method": "tools/call",
  "params": {
    "name": "execute_sql",
    "arguments": {
      "query": "SELECT * FROM customers WHERE email='alice@example.com'"
    },
    "_meta": {
      "progressToken": "abc123"
```

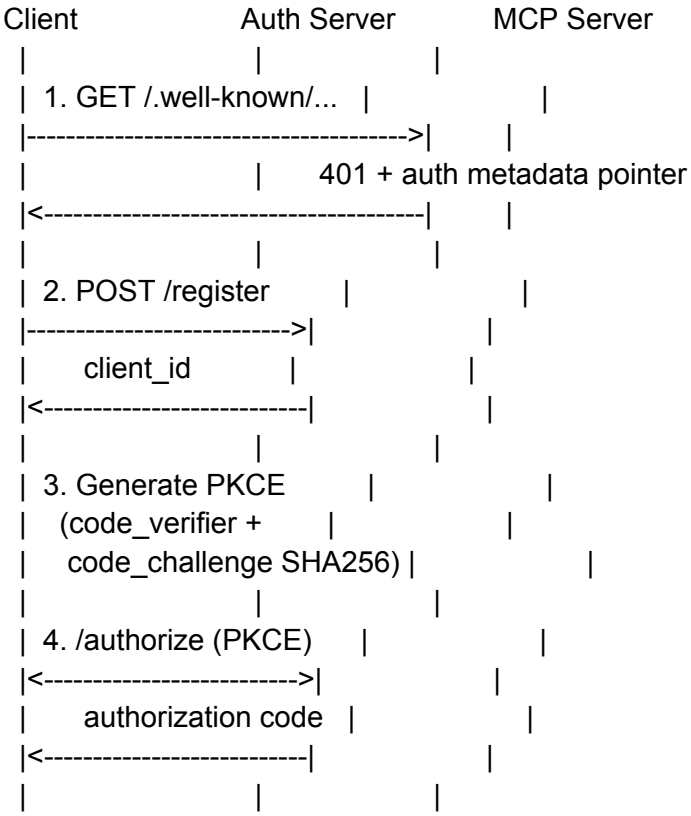
```
  }
 }
}
```

What you log:

- `agent_id` (which AI client)
- `tool_name` ("execute_sql")
- `parameters` (the full arguments object)
- `request_id` (42)
- `timestamp` (when received)
- `response` (full response from server)
- `duration_ms` (how long it took)
- `status` ("allowed" / "blocked" / "flagged")
- `policy_violations` (if any rules triggered)
- `pii_detected` (boolean + types found)

The OAuth 2.1 + PKCE flow (for HTTP MCP servers)

This is the production auth standard. You will need to support this. Verified flow from MCP spec:



```
|  5. /token (code+verifier) |                    |
|--------------------------->|                    |
|      access_token          |                    |
|<---------------------------|                    |
|                            |                    |
|  6. POST /mcp + Bearer token                    |
|------------------------------------------------>|
|                            |     200 OK + result |
|<------------------------------------------------|
```

The 4 RFCs you must implement:

- RFC 9728 — Protected Resource Metadata
- RFC 8707 — Resource Indicators (binds tokens to specific MCP server)
- RFC 8414 — Authorization Server Metadata
- RFC 7591 — Dynamic Client Registration

Critical security requirements (from MCP spec, NON-NEGOTIABLE):

- PKCE with SHA-256 (plain method NOT allowed under OAuth 2.1)
- HTTPS for all authorization endpoints
- Redirect URIs must be localhost OR HTTPS only
- Token audience validation (token only valid for the specific MCP server)
- Token isolation (don't pass tokens to other servers)
- Bearer token in `Authorization` header (NEVER in URL)

The CVEs you need to know

These are real, current vulnerabilities. You'll be asked about them by sophisticated buyers.

CVE-2025-6514 — `mcp-remote` npm package, RCE via OAuth flow injection. 437,000+ developer environments compromised. Caused by metadata fields not being sanitized. **Lesson:** Sanitize EVERY field from metadata discovery.

CVE-2025-49596 — Unauthenticated command injection. CVSS 9.4. **Lesson:** Validate all parameters before forwarding.

CVE-2025-53110 — Filesystem MCP sandbox escape. **Lesson:** Path traversal protection on file MCP servers.

CVE-2025-53109 — Symlink bypass on filesystem MCP. **Lesson:** Resolve symlinks before validation.

PART 3: THE COMPLETE PRODUCT — EVERY FEATURE

Layer 1: Free CLI Scanner (Acquisition Hook)

What it is

A single npm command (`npx mcp-security-scan`) that scans a developer's machine and outputs a security report on every MCP server they have configured.

What it scans

1. Claude Desktop config (`~/Library/Application Support/Claude/` on Mac, `%APPDATA%\Claude\` on Windows)
2. Cursor config (`~/.cursor/mcp.json`)
3. VS Code config (`~/.vscode/`, workspace settings)
4. Continue.dev config
5. Cline config
6. Windsurf config
7. Currently running processes (find any node/python processes that look like MCP servers)
8. npm/pip globally installed packages matching known MCP package patterns

What it checks for each server

1. **Auth status** — Does it use OAuth or static API keys? (most use static keys = risk)
2. **Hardcoded secrets** — Scan config args/env for patterns matching API keys, AWS credentials, database passwords
3. **Tool inventory** — Connect to the server and call `tools/list` — show user every tool exposed
4. **Permission scope** — For known servers, show what they can access (filesystem, GitHub, DBs, etc.)
5. **Server reputation** — Compare package name/source to known malicious patterns
6. **CVE matches** — Check installed versions against known vulnerabilities

Sample output

🔍 MCP Security Scan Report
Generated: 2026-05-01 14:23:01

Found 12 MCP servers configured.

Detected: 8 high-risk, 3 medium-risk, 1 low-risk.

🔴 HIGH RISK

1. server-postgres (Cursor)
   ⚠️ Hardcoded database credentials in config
   ⚠️ Has SELECT/INSERT/UPDATE/DELETE on all tables
   ⚠️ No audit log of queries
   Path: ~/.cursor/mcp.json

2. github-mcp-server (Claude Desktop)
   ⚠️ Personal access token in env var (no rotation)
   ⚠️ Token has admin scope
   ⚠️ CVE-2025-6514 vulnerability (mcp-remote 1.2.0)
   Path: ~/Library/Application Support/Claude/

[8 more servers...]

📊 Summary
Servers with hardcoded credentials: 9 (75%)
Servers without OAuth: 11 (92%)
Servers without audit logging: 12 (100%)

🛡️ Want to monitor and control these in production?
Try the managed gateway: https://[your-domain]/start

Build effort

3-5 days. Pure JavaScript/TypeScript. No backend needed.

Distribution

- Open source on GitHub
- Submit to Hacker News (Show HN format)
- Tweet thread with screenshots
- LinkedIn post showing real findings
- DEV.to article walking through the code
- Eventually: integrate with `npm audit`, GitHub security scanning

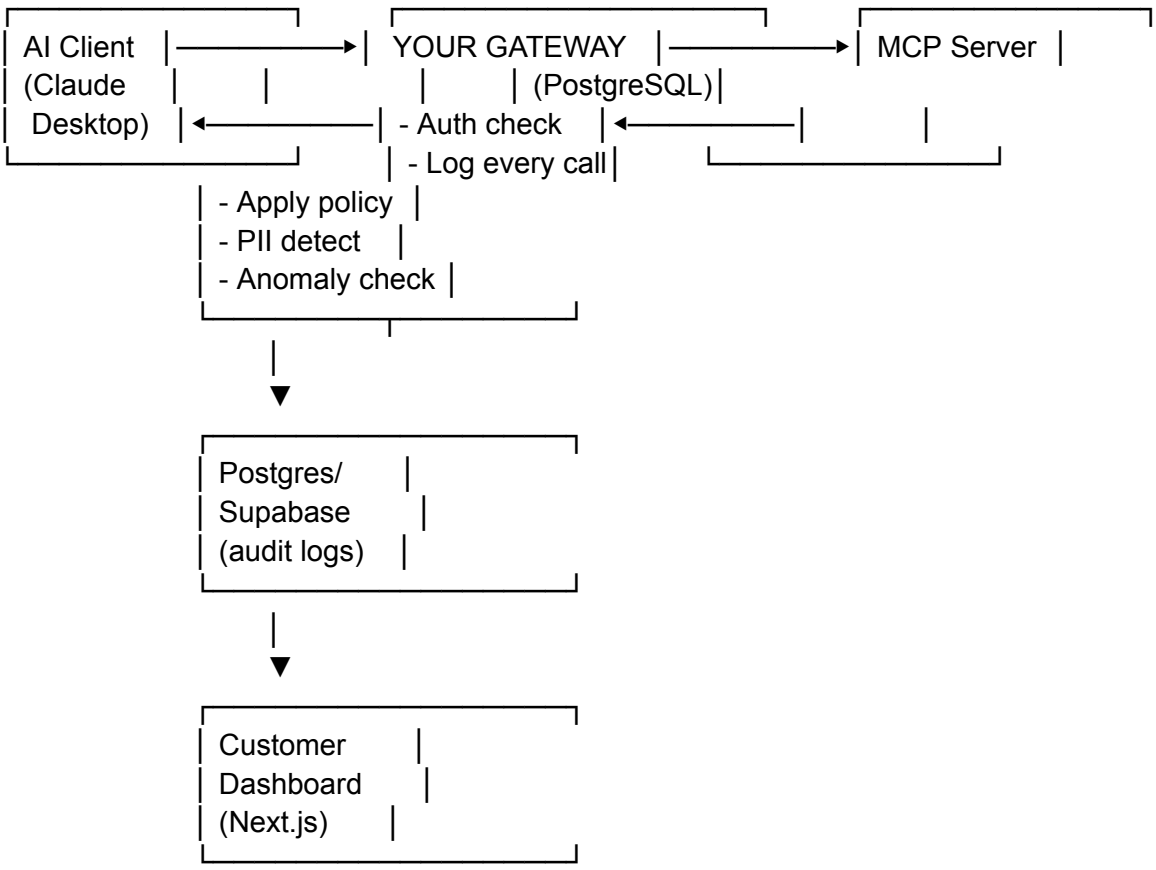
The conversion path

Free scan → User sees they have problems → CTA in output → Sign up for managed gateway

---

Layer 2: The Gateway Proxy (Core Product)

Architecture overview



Two gateway modes (you build both, customers pick)

Mode A: Local proxy (stdio) What you already built today. Customer adds a line to their `mcp.json`:

```
{
  "mcpServers": {
    "their-server": {
      "command": "node",
      "args": [
        "/path/to/your-gateway",
```

```
        "--api-key", "their-key",
        "--upstream-command", "node",
        "--upstream-args", "their-actual-server.js"
      ]
    }
  }
}
```

Your gateway spawns the real server as a subprocess, intercepts everything between Claude Desktop and the server.

Time to build: Already done. Just need to clean up.

Mode B: Remote HTTP gateway Customer points their AI client to your hosted endpoint:

```
{
  "mcpServers": {
    "their-server": {
      "url": "https://gateway.your-domain.com/their-server-id",
      "headers": {
        "Authorization": "Bearer their-access-token"
      }
    }
  }
}
```

You proxy to their actual MCP server (which they hosted somewhere), or to a managed instance you run for them.

Time to build: 7-10 days.

What the gateway does on every request

```
// pseudocode
async function handleToolCall(request) {
  const startTime = Date.now();

  // Step 1: Authenticate the caller
  const auth = await authenticateRequest(request);
  if (!auth.valid) return reject(401);

  // Step 2: Look up the customer's policies
  const policies = await getPoliciesForCustomer(auth.customerId);
```

```
  // Step 3: Pre-execution checks
  const piiDetection = await detectPII(request.params.arguments);
  const policyViolations = checkPolicies(request, policies);
  const anomalyScore = checkAnomalies(request, auth.customerId);

  // Step 4: Block if policy says so
  if (policyViolations.length > 0 && policies.blockOnViolation) {
    await logBlockedRequest(request, policyViolations);
    return rejectRequest("Policy violation: " + policyViolations[0].name);
  }

  // Step 5: Forward to upstream MCP server
  const response = await forwardToMCPServer(request);

  // Step 6: Post-execution checks (response also has PII)
  const responsePII = await detectPII(response);

  // Step 7: Log everything
  await logToolCall({
    customerId: auth.customerId,
    agentId: auth.agentId,
    toolName: request.params.name,
    parameters: request.params.arguments,
    response: response,
    durationMs: Date.now() - startTime,
    piiDetected: [...piiDetection, ...responsePII],
    policyViolations: policyViolations,
    anomalyScore: anomalyScore,
    status: policyViolations.length > 0 ? "blocked" : "allowed",
    timestamp: new Date()
  });

  // Step 8: Return response (possibly redacted)
  return policies.redactPII ? redactPII(response, responsePII) : response;
}
```

Database schema you need

```
-- Your existing table, expanded
CREATE TABLE tool_calls (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  created_at TIMESTAMPTZ DEFAULT NOW(),

  -- Tenant + agent
```

```
  customer_id UUID NOT NULL REFERENCES customers(id),
  agent_id TEXT NOT NULL,
  session_id UUID,

  -- The call itself
  server_name TEXT NOT NULL,
  tool_name TEXT NOT NULL,
  parameters JSONB,
  request_id TEXT,

  -- Response
  response JSONB,
  duration_ms INTEGER,

  -- Security signals
  status TEXT CHECK (status IN ('allowed', 'blocked', 'flagged', 'error')),
  policy_violations JSONB,
  pii_detected JSONB,
  anomaly_score FLOAT,

  -- Source tracking
  source_ip TEXT,
  user_agent TEXT
);

CREATE INDEX idx_tool_calls_customer ON tool_calls(customer_id, created_at DESC);
CREATE INDEX idx_tool_calls_status ON tool_calls(customer_id, status);
CREATE INDEX idx_tool_calls_tool ON tool_calls(customer_id, tool_name);

-- Customers
CREATE TABLE customers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email TEXT UNIQUE NOT NULL,
  company_name TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW(),

  api_key TEXT UNIQUE NOT NULL,
  plan TEXT DEFAULT 'free',

  -- Stripe
  stripe_customer_id TEXT,
  stripe_subscription_id TEXT,
  trial_ends_at TIMESTAMPTZ
);
```

```
-- Policies (rules)
CREATE TABLE policies (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  customer_id UUID NOT NULL REFERENCES customers(id),
  created_at TIMESTAMPTZ DEFAULT NOW(),

  name TEXT NOT NULL,
  description TEXT,
  enabled BOOLEAN DEFAULT TRUE,

  -- Rule logic stored as JSON
  rule_type TEXT, -- 'block_tool', 'block_param_pattern', 'rate_limit', etc.
  rule_config JSONB,

  action TEXT CHECK (action IN ('block', 'flag', 'alert')),
  alert_channels JSONB -- ['email', 'slack', etc.]
);

-- MCP servers being protected
CREATE TABLE mcp_servers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  customer_id UUID NOT NULL REFERENCES customers(id),
  name TEXT NOT NULL,
  transport TEXT CHECK (transport IN ('stdio', 'http')),
  upstream_config JSONB, -- command/args for stdio, URL for http
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Sessions (track an AI client's session)
CREATE TABLE sessions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  customer_id UUID NOT NULL REFERENCES customers(id),
  agent_id TEXT NOT NULL,
  started_at TIMESTAMPTZ DEFAULT NOW(),
  ended_at TIMESTAMPTZ,
  total_calls INTEGER DEFAULT 0
);
```

Layer 3: The Dashboard (How Customers See Value)

Pages to build

1. Live Activity Feed (`/dashboard`) The home page. Real-time stream of tool calls.

Top section — Summary cards:

- Total tool calls (last 24h)
- Blocked calls
- PII detections
- Active agents
- MCP servers monitored

Main section — Live table:

```
Time     | Agent       | Server    | Tool          | Status   | Actions
14:23:01 | claude-prod | postgres  | execute_sql   | Allowed  | View
14:23:04 | claude-prod | gmail     | send_email    | Blocked  | View (PII)
14:22:58 | cursor-dev  | filesys   | read_file     | Allowed  | View
14:21:33 | claude-prod | postgres  | delete_record | Flagged  | Approve
```

Each row clicks into a detail drawer showing full request/response, parameters, PII detected, etc.

2. Tool Calls Browser (`/dashboard/calls`) Searchable, filterable history. SQL-style filters.

Filters:

- Date range
- Agent ID
- Tool name
- Status
- Has PII
- MCP server

Each call has detail page with:

- Full request payload
- Full response payload
- PII matches highlighted
- Policy evaluation result
- Anomaly score breakdown

3. Compliance Reports (`/dashboard/reports`) The killer feature for closing enterprise deals.

One-click PDF export with:

- Cover page (customer logo, date range)
- Executive summary (total calls, blocked, PII redacted)
- Authentication summary (OAuth status, token rotation)
- Detailed audit trail (paginated table of all calls)
- Policy compliance summary
- Encryption details (TLS 1.3 in transit, AES-256 at rest)
- Retention policy

This is what customers send to enterprise security teams. THIS feature alone justifies $299/month.

4. Policies (`/dashboard/policies`) List of active rules. Toggle on/off. Add new rules.

Pre-built rule templates:

- "Block any tool call containing credit card numbers"
- "Block any tool call between 10pm-8am UTC"
- "Rate limit any tool to 100 calls/hour"
- "Flag any DELETE operations"
- "Block tool calls from agents not in allowlist"
- "Require approval for tool X"

Each rule:

- Type
- Conditions (configurable)
- Action (block, flag, alert)
- Alert channels (email, Slack)

5. MCP Servers (`/dashboard/servers`) List of MCP servers being protected. Add new ones.

Per server:

- Name
- Transport (stdio/HTTP)
- Connection status
- Tool calls today
- Health check status
- Tool inventory (what tools it exposes)

6. Agents (`/dashboard/agents`) List of AI clients/agents calling through your gateway.

Per agent:

- ID (e.g., "claude-desktop", "cursor-vscode")
- First seen
- Last active
- Total calls
- Blocked rate
- Tools used most

7. Settings (`/dashboard/settings`)

- API keys (rotate, revoke)
- Team members (invite, RBAC — later)
- Billing (Stripe customer portal)
- Integrations (Slack, email alerts, webhooks)
- Data retention

Tech stack for dashboard

- **Frontend:** Next.js 15 (you know this) + Tailwind + shadcn/ui
- **Auth:** Clerk or Supabase Auth (don't build your own)
- **Realtime updates:** Supabase Realtime subscriptions
- **PDF generation:** `@react-pdf/renderer` or `pdfkit`
- **Charts:** Recharts (built into shadcn)
- **Tables:** TanStack Table

Build effort

10-14 days for v1 dashboard. Not 30. You can ship it.

Layer 4: Policy Engine

What it is

A rules engine that evaluates every tool call against configured policies before execution. Blocks, flags, or alerts.

Rule types to support (in priority order)

Tier 1 — Build first (Day 1 of policy engine):

1. **Block tool by name** — Block `delete_record` always
2. **Block tool by pattern** — Block any tool starting with `admin_`
3. **Allow only listed tools** — Whitelist mode

4. **Rate limit per tool** — Max 100 calls/hour for `send_email`
5. **Block by parameter pattern** — Block calls where SQL contains `DROP`

Tier 2 — Build after first customers:

6. **Time-of-day restrictions** — No destructive ops between 10pm-8am
7. **Require approval** — Some calls require human Slack approval
8. **Block by parameter PII** — Block if any parameter contains SSN
9. **Block external destinations** — Email tool can't send to non-corporate domains
10. **Require comment** — Calls with sensitive operations need a justification field

Tier 3 — Premium features:

11. **Anomaly-based blocking** — Block if anomaly score > threshold
12. **Cross-tool patterns** — Detect when agent reads sensitive data then calls external API
13. **Custom JavaScript rules** — Customer writes their own evaluator

Implementation approach

Don't over-engineer. Use simple JSON rule definitions:

```
type Rule = {
  id: string;
  name: string;
  type: 'block_tool' | 'block_param_regex' | 'rate_limit' | 'block_time_range';
  enabled: boolean;

  // Type-specific config
  config: {
    // For block_tool
    tool_names?: string[];

    // For block_param_regex
    param_path?: string; // e.g., "arguments.query"
    pattern?: string; // regex

    // For rate_limit
    tool_name?: string;
    max_calls?: number;
    window_seconds?: number;

    // For block_time_range
    start_hour?: number;
    end_hour?: number;
    timezone?: string;
```

```
  };

  action: 'block' | 'flag' | 'alert';
  alert_channels?: ('email' | 'slack' | 'webhook')[];
};
```

Evaluator runs all enabled rules for the customer in sequence. First matching rule with `block` action stops execution.

Build effort

5-7 days for Tier 1.

Layer 5: PII Detection

What it does

Scans every parameter and response for sensitive data. Flags or redacts.

What it detects (in priority order)

Pattern-based (regex, fast):

1. Credit card numbers (Luhn check)
2. SSN (US)
3. Phone numbers
4. Email addresses (sometimes PII, sometimes not)
5. IP addresses (sometimes)
6. AWS keys (`AKIA...`)
7. Stripe keys (`sk_live_...`, `pk_live_...`)
8. GitHub tokens (`ghp_...`, `gho_...`)
9. Slack tokens (`xoxb-...`, `xoxp-...`)

LLM-based (Claude API call, slower but smarter): 10. Names (when in customer context) 11. Addresses 12. Medical information 13. Financial account numbers 14. Custom PII types per customer

Implementation

Phase 1 — Regex only (fast, ship first):

Use the `presidio-analyzer` library or build your own regex set. Sub-millisecond detection.

```
const PII_PATTERNS = {
  credit_card: /\b(?:4\d{3}|5[1-5]\d{2}|6011|3[47]\d{2})[-\s]?\d{4}[-\s]?\d{4}[-\s]?\d{4}\b/g,
  ssn: /\b\d{3}-\d{2}-\d{4}\b/g,
  aws_key: /\bAKIA[0-9A-Z]{16}\b/g,
  stripe_secret: /\bsk_live_[0-9a-zA-Z]{24,}\b/g,
  github_token: /\bghp_[0-9a-zA-Z]{36}\b/g,
  email: /\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b/g,
};

function detectPII(text: string): PIIMatch[] {
  const matches = [];
  for (const [type, pattern] of Object.entries(PII_PATTERNS)) {
    const found = text.match(pattern) || [];
    for (const match of found) {
      // Validate (e.g., Luhn check for credit cards)
      if (validate(type, match)) {
        matches.push({ type, value: match, position: text.indexOf(match) });
      }
    }
  }
  return matches;
}
```

Phase 2 — LLM augmentation (premium tier):

Send tool call to Claude API with a prompt asking it to identify PII. Cache results aggressively.

Redaction modes

- **Detect only** — Log the PII, don't modify request
- **Mask** — Replace `4111-1111-1111-1234` with `4111-XXXX-XXXX-1234`
- **Hash** — Replace with SHA-256 hash (preserves uniqueness for analytics)
- **Block** — Reject the entire request

Build effort

3-5 days for regex-based. Add LLM later.

Layer 6: OAuth 2.1 Authentication

When you need this

Not on Day 1. stdio MCP servers don't need OAuth. Your first 5-10 customers will probably be on stdio.

You need it when:

- A customer wants to put your gateway in front of HTTP MCP servers
- A customer wants different agents to have different permissions
- An enterprise prospect asks "how do you authenticate agents?"

What you build

You become an OAuth 2.1 authorization server. Implement these endpoints:

POST /.well-known/oauth-authorization-server
  Returns metadata document (RFC 8414)

POST /register
  Dynamic client registration (RFC 7591)

GET /authorize
  PKCE-protected authorization flow

POST /token
  Token issuance with PKCE verification

POST /introspect
  Token introspection (RFC 7662)

POST /revoke
  Token revocation

Don't build this yourself

Use a managed OAuth provider. Verified options that work for MCP in 2026:

- **Scalekit** — Drop-in OAuth specifically for MCP. Recommended.
- **Auth0** — Enterprise-grade, expensive
- **Clerk** — Newer, simpler, good for B2B
- **Supabase Auth** — Free tier, you already use Supabase
- **Keycloak** — Self-hosted, free, complex

My recommendation: Use Supabase Auth for your DASHBOARD authentication. Use Scalekit (or implement minimal OAuth flow with `mcp` SDK helpers) for MCP-level OAuth.

The flow customers go through

1. Customer adds an HTTP MCP server to your gateway
2. Customer's AI client connects to your gateway URL
3. Gateway responds 401 with metadata pointer
4. AI client discovers OAuth endpoints via `.well-known`
5. AI client registers as OAuth client (dynamic registration)
6. AI client does PKCE auth flow
7. Gets access token bound to your gateway URL
8. Sends every request with `Authorization: Bearer <token>`
9. Gateway validates token, identifies which agent/user
10. Gateway logs and forwards to upstream MCP server

Build effort

If you use Scalekit or similar: 5-7 days. If you build from scratch: 3-4 weeks.

Recommendation: Don't build from scratch. Use Scalekit. Their entire pitch is "drop in OAuth for MCP in minutes."

Layer 7: Anomaly Detection

What it does

Looks at patterns of tool calls and flags unusual behavior. NOT for v1. Build after first customers.

Approaches (cheap to expensive)

Approach 1: Statistical baselines (cheap, ship first) Per agent, track:

- Average calls per hour
- Most common tools
- Average parameter sizes
- Common destinations

Flag if current behavior deviates >3 standard deviations.

Approach 2: Sequence patterns (medium) Track common tool call sequences. Flag novel sequences.

Approach 3: Claude API analysis (expensive but smartest) Send recent tool call history to Claude API with prompt:

> "Below is the recent tool call history for an AI agent. Is anything unusual or potentially malicious? Score 0-100."

Build effort

10-14 days for Approach 1. Don't ship until you have customers asking.

Layer 8: Billing & Self-Serve Signup

Stripe integration

Use Stripe Checkout (not custom payment forms). Subscription tiers:

```
const TIERS = {
  free: {
    name: 'Free',
    price: 0,
    limits: {
      tool_calls_per_day: 100,
      log_retention_days: 7,
      mcp_servers: 1,
      policies: 0,
      pii_detection: false,
      pdf_export: false,
    }
  },
  starter: {
    name: 'Starter',
    price: 299,
    stripe_price_id: 'price_xxx',
    limits: {
      tool_calls_per_day: 10000,
      log_retention_days: 30,
      mcp_servers: 3,
      policies: 5,
      pii_detection: true,
```

```
      pdf_export: true,
    }
  },
  growth: {
    name: 'Growth',
    price: 799,
    stripe_price_id: 'price_yyy',
    limits: {
      tool_calls_per_day: 100000,
      log_retention_days: 90,
      mcp_servers: 10,
      policies: 25,
      pii_detection: true,
      pdf_export: true,
      slack_alerts: true,
      anomaly_detection: 'basic',
    }
  },
  scale: {
    name: 'Scale',
    price: 2000,
    stripe_price_id: 'price_zzz',
    limits: {
      tool_calls_per_day: -1, // unlimited
      log_retention_days: 365,
      mcp_servers: -1,
      policies: -1,
      pii_detection: true,
      pdf_export: true,
      slack_alerts: true,
      anomaly_detection: 'advanced',
      api_access: true,
      priority_support: true,
    }
  }
};
```

Signup flow

1. Land on pricing page
2. Click "Start free trial" on any paid tier
3. Email + password signup (Supabase Auth)
4. Stripe Checkout for billing details (14-day free trial, no charge)

5. Generate API key
6. Show "Get started" page with one-line install command
7. Run scanner first (warm-up)
8. Set up first MCP server
9. Show first tool calls flowing through dashboard

The "aha moment" target: Signup → first tool call logged in dashboard in under 10 minutes.

Build effort

3-5 days for Stripe + signup flow.

PART 4: WHAT YOU NEED — COMPLETE DEPENDENCY LIST

Services & Subscriptions (start of build)

Service	Purpose	Cost
Vercel	Frontend hosting	Free → $20/mo
Supabase	Database + Auth + Storage	Free → $25/mo
Stripe	Payments	2.9% + 30¢ per txn
Resend or Postmark	Transactional email	Free → $15/mo
Sentry	Error tracking	Free → $26/mo
GitHub	Code hosting	Free
Domain	Your branded domain	$12/year
Cloudflare	DNS + CDN	Free

Monthly burn at start: ~$50-100/month. Sustainable.

Optional but recommended

Service	Purpose	Cost
Apollo.io	Lead enrichment for outbound	$99/mo
Instantly.ai	Cold email automation	$37/mo
Loom	Demo video recording	Free → $15/mo
Cal.com	Booking demo calls	Free
PostHog	Product analytics	Free → $0/mo at low volume
Clerk or Scalekit	OAuth for MCP	Free → $25/mo

Tech stack (no debate, just pick these)

Frontend:

- Next.js 15 (App Router)
- TypeScript
- Tailwind CSS
- shadcn/ui (component library)
- TanStack Query (data fetching)
- TanStack Table (data tables)
- Recharts (charts)
- Zustand or React Context (state)

Backend:

- Node.js 20+
- Hono or Express (API framework)
- TypeScript
- Supabase JS SDK
- @modelcontextprotocol/sdk (official MCP SDK)
- Zod (schema validation)

Database:

- PostgreSQL via Supabase
- Use `pgcrypto` for token hashing
- Row-level security (RLS) for multi-tenancy

Infrastructure:

- Vercel for Next.js + serverless functions
- Vercel Postgres or Supabase Postgres

- Cloudflare in front of everything (DDoS protection, WAF)

Why not other options?

- Don't use SvelteKit/Remix — smaller community, fewer libs
- Don't use AWS Lambda directly — Vercel is easier and same underneath
- Don't self-host PostgreSQL — Supabase is fine, free tier is generous
- Don't use MongoDB — JSON queries on Postgres are fine

Skills you need

You said you know:

- Next.js ✅
- TypeScript ✅
- Python ✅ (won't need much for this)
- Claude API ✅

Skills you'll need to learn (in priority order):

1. **Postgres advanced features** — Indexes, JSON queries, RLS. (1-2 days of learning)
2. **OAuth 2.1 implementation** — Even if using Scalekit, understand the flow. (3-5 days reading)
3. **Streaming HTTP / SSE** — For remote MCP. (2-3 days)
4. **Stripe Checkout + webhooks** — Subscription handling. (2 days)

What you don't need

- Don't need React Native (no mobile app)
- Don't need GraphQL (REST is fine)
- Don't need Kubernetes (Vercel handles deployment)
- Don't need a separate auth service (Supabase Auth is fine for users)
- Don't need Redis caching (Postgres is fast enough at this scale)
- Don't need monitoring beyond Sentry
- Don't need TimescaleDB (regular Postgres + indexes are fine until 100M+ rows)

PART 5: THE 30-DAY BUILD PLAN

Week 1 (Days 1-7): Solidify Foundation

You have: Working stdio gateway logging to Supabase. ✅

This week:

Day 1 (today/tomorrow):

- Clean up `proxy.mjs` — proper error handling, response logging, duration tracking
- Add response logging to schema and code
- Test with multiple MCP servers (not just Sevlex)

Day 2-3:

- Set up new Next.js project for dashboard
- Implement Supabase Auth (signup, login, magic links)
- Basic protected routes structure

Day 4-5:

- Live activity feed page (`/dashboard`)
- Real-time updates via Supabase Realtime
- Tool call detail drawer

Day 6-7:

- Tool calls history page (`/dashboard/calls`)
- Filters (date, agent, tool, status)
- Search functionality

Goal at end of Week 1: Customer can sign up, get API key, install gateway, see real-time activity in dashboard.

Week 2 (Days 8-14): Free Scanner + Public Launch Prep

Day 8-10:

- Build CLI scanner (`mcp-security-scan` npm package)
- Scans Claude/Cursor/VS Code configs
- Detects hardcoded secrets, OAuth status, tool inventory
- Open source on GitHub

Day 11:

- PDF compliance export feature
- Use `@react-pdf/renderer`
- Customer logo, branding, executive summary

Day 12-13:

- Public landing page
- Pricing page (4 tiers, public)
- Comparison page (vs Runlayer, vs MintMCP)
- Documentation site (Mintlify or Nextra)

Day 14:

- Hacker News submission day (Tuesday or Wednesday 8am PT)
- Title: "Show HN: Free CLI to find shadow MCP servers in your environment"
- Be present in comments for 4 hours

Goal at end of Week 2: First 100 GitHub stars on scanner. First 10 trial signups from HN.

Week 3 (Days 15-21): Policy Engine + Stripe

Day 15-17:

- Policy engine implementation
- Tier 1 rules: block_tool, block_param_regex, rate_limit, block_time_range
- Policies UI in dashboard
- Pre-built rule templates

Day 18-19:

- Stripe integration
- Checkout flow
- Subscription webhook handling
- Plan limit enforcement
- Customer portal (Stripe-hosted)

Day 20-21:

- PII detection (regex-based)
- Show PII matches in dashboard
- PII redaction option in policies

Goal at end of Week 3: Product is fully self-serve. Customer can sign up, pay, configure policies, see PII detection working.

Week 4 (Days 22-30): Polish + First Customer

Day 22-23:

- Slack alert integration
- Email alert integration
- Webhook support

Day 24-25:

- Onboarding flow (interactive setup wizard)
- Email sequence (Day 0, 1, 3, 7, 12, 14)
- "Aha moment" optimization

Day 26-27:

- Documentation: getting started, OAuth setup, policy examples
- Demo video (Loom, 3 minutes)
- Comparison content (Runlayer alternative, MintMCP alternative)

Day 28-30:

- Outbound to ICP list (50 messages/day)
- First demo calls
- Close first customer

Goal at end of Day 30: First paying customer. $299 MRR.

PART 6: WHAT TO BUILD AFTER 30 DAYS

Month 2 (Days 31-60)

Higher-priority features:

- OAuth 2.1 server (use Scalekit or implement minimal version)
- Remote HTTP MCP gateway support
- Anomaly detection (statistical approach)
- Team management / RBAC

Distribution:

- 200 outbound emails/week
- Daily LinkedIn content
- Hacker News Launch #2: "I scanned 1,000 MCP servers, here's what I found"
- First case study (from first customer)

Goal: 5 paying customers. $2,500-5,000 MRR.

Month 3 (Days 61-90)

Features:

- Advanced policy rules (time-based, cross-tool patterns)
- Custom PII patterns per customer
- API access for paid tiers
- Audit log search via API

Distribution:

- Hacker News Launch #3: full product launch
- "Self-serve alternative to Runlayer"
- Multiple comparison posts ranking on Google
- YC Fall 2026 application submitted

Goal: 10+ paying customers. $5,000-10,000 MRR. YC interview booked.

PART 7: WHAT TO BUILD AFTER YC

After YC funding (assume Series A in 2027):

Enterprise features:

- SSO / SAML
- Dedicated VPC deployment
- SOC 2 Type II certification (start at month 6, finishes ~month 18)
- HIPAA compliance
- Custom data retention policies
- Dedicated Slack channel support
- SLA guarantees (99.95%+)

Advanced features:

- ML-based threat detection (like Runlayer's ToolGuard)
- MCP server registry/catalog
- Tool call sandboxing
- Multi-region deployment

Don't build any of this before YC. Series A buyers don't need it. Solo founders can't build it. It's a trap.

PART 8: COMMON MISTAKES TO AVOID

Technical mistakes

1. Logging tokens in audit trail Never. Log tool name, parameters (with PII redacted), and response shape. Don't log raw OAuth tokens — if your DB is breached, attacker has all customer credentials.

2. Putting tokens in URLs Per OAuth 2.1 spec, NEVER. Always use `Authorization: Bearer` header.

3. Not validating token audience Each token must be bound to YOUR gateway URL via the `resource` parameter. If you accept tokens issued for any resource, attacker can use a stolen token from elsewhere.

4. Shipping without rate limiting Single misbehaving customer can DDoS your service. Implement per-customer rate limits from Day 1.

5. Not sanitizing metadata fields CVE-2025-6514 happened because of unsanitized metadata fields. Sanitize EVERYTHING from external sources.

6. Storing customer data without RLS Use Supabase Row-Level Security from the start. One bug, customers see each other's data, you're done.

7. No backup strategy Audit logs are valuable. Backup daily. Test restore quarterly.

8. Building OAuth 2.1 from scratch Use Scalekit. The MCP spec compliance is hard. Scalekit specifically builds for MCP.

Product mistakes

9. Adding too many features before customers Ship what's in this doc. Stop. Get customers. Let them tell you what's missing.

10. Free tier that's too generous Give scanner free. Give 100 tool calls/day free. Don't give 10,000 free — paying tier shouldn't be 100x more useful.

11. Pricing too low $299 is right. Don't price at $49 because "you're new." Series A companies can pay $299. They can't pay $49 — feels like a hobby project.

12. Selling to wrong ICP Don't sell to:

- Pre-seed startups (no budget)
- F500 enterprises (too slow, need SOC 2)
- Individual developers (use free tier forever)

DO sell to:

- Series A AI-native companies, 50-300 employees, CTO buying

13. Trying to win on features vs Runlayer You can't. Win on access (self-serve, public pricing, 30-min deploy).

14. Hiring too early Don't hire anyone until $20k+ MRR. Then hire one person — engineer, not salesperson.

Distribution mistakes

15. Cold messaging without content first Per 2026 LinkedIn data: cold = 1.7% conversion. Content + warm = 14.6%. Build content presence before outbound.

16. Hiding pricing Every competitor does this. Don't. Public pricing is your differentiator.

17. Long sales calls You don't have time. Self-serve onboarding. Async demos via Loom.

18. Not measuring Track: signups/week, free→paid conversion, MRR, churn. Weekly. In a spreadsheet.

PART 9: ONE FINAL CHECKLIST

Before you start tomorrow, verify you have:

Accounts:

- [ ] Vercel account
- [ ] Supabase account (you already have this)
- [ ] Stripe account (start in test mode)
- [ ] GitHub account
- [ ] Domain registered (e.g., trysevlex.com or get-mcpsec.com)

- [ ] Cloudflare account (point domain DNS here)
- [ ] Resend or Postmark for emails
- [ ] PostHog for analytics (optional but recommended)

Repositories:

- [ ] `mcp-security-gateway-proxy` (your gateway code)
- [ ] `mcp-security-gateway-dashboard` (Next.js app)
- [ ] `mcp-security-scan` (CLI scanner — public, open source)

Documents:

- [ ] Privacy policy (use Termly or RocketLawyer)
- [ ] Terms of service
- [ ] Acceptable use policy
- [ ] Security policy (one-page summary you can share)

Communication channels:

- [ ] Customer support email (`support@yourdomain.com`)
- [ ] Sales email (`hi@yourdomain.com`)
- [ ] Twitter handle
- [ ] LinkedIn profile updated
- [ ] Cal.com link for demo bookings

PART 10: THE NORTH STAR

You have a working product foundation today. Most founders don't.

You have proven outreach skills. Most founders don't.

You have 6 hours/day of focused work time. Most founders don't.

You have a real, validated market. The 30+ competitors prove it. The Vercel breach proves it. The Felicis blog post on Runlayer proves it.

You have a real, identifiable gap. Self-serve at $299 for Series A AI startups. Nobody is sitting in that seat.

What you don't have:

- A famous co-founder (don't need one for first 10 customers)

- $11M (don't need it — your tech costs are <$100/month)
- A US network (build it through content, not through introductions)
- SOC 2 (don't need it for Series A buyers — they don't ask)

The single number you track for the next 90 days: paying customers.

Not MRR. Not signups. Not GitHub stars. Not LinkedIn followers.

Paying customers.

10 of them by Day 90. Apply YC.

Everything else in this document supports that one number.

Build the dashboard tomorrow. Send the messages. Ship the scanner. Launch on Hacker News.

The product is real. The market is real. The path is clear.

Now go execute.

END OF DOCUMENT

This is your A-to-Z product playbook. Reference it when you're stuck. Update it as you learn. Don't deviate without good reason.

The first 10 customers will tell you 80% of what to build next. Until then, this document is the truth.

MCP SECURITY GATEWAY — COMPLETE PRODUCT PLAYBOOK

Prepared for: Meet Panchal | Founder **Status:** Day 1 build complete (Sevlex MCP gateway logging to Supabase confirmed working) **Goal:** Build complete product to charge first customer in 30 days **Date:** May 1, 2026 **Source quality:** Only verified facts — official MCP spec at modelcontextprotocol.io, real OAuth 2.1 / PKCE implementation guides from 2026, competitor docs, your own working code. No assumptions. No bluff.

READ THIS FIRST

This document is the complete A-to-Z product specification. Read it once front to back. Then keep it open while building.

You already shipped the hardest part on Day 1: A working stdio MCP proxy that intercepts every tool call and logs to Supabase. That's the foundation everything else stands on.

What's left is execution work, not invention work:

- Build the dashboard
- Add OAuth for remote MCP servers
- Add policy engine
- Add PII detection
- Ship a free shadow scanner
- Wire up Stripe billing
- Launch publicly

Every feature in this document has been built before by someone else. None of it requires research breakthroughs. It requires you to sit down and build for 30 days.

PART 1: WHAT THIS PRODUCT ACTUALLY IS

In one paragraph

You are building a **security gateway for the Model Context Protocol (MCP)**. MCP is the standard Anthropic released in November 2024 that lets AI agents call external tools (databases, emails, APIs, file systems). Your gateway sits between the AI client and the MCP servers — every tool call passes through it. You authenticate, log, inspect, and control every call. Without your gateway, the AI agent has uncontrolled access to everything. With your gateway, every action is identified, recorded, and bound by policy.

In one sentence to a customer

> "We sit between your AI agents and the tools they use, so you know exactly what your AI did, can prove it to enterprise buyers, and can stop it from doing something dangerous."

The actual problem you solve (in customer's words)

A 50-300 person AI startup has built an AI agent product. The agent uses MCP servers to access databases, send emails, write files, call APIs. Three customer scenarios drive purchase:

Scenario 1 — Enterprise deal in progress: Their enterprise prospect's security team sent a 47-page security questionnaire. Question 23: "How do you audit every action your AI agent takes?" The startup has no answer. Deal stalls.

Scenario 2 — Already had a near-miss: An engineer connected their AI agent to a production MCP server with admin privileges. The agent accidentally ran a destructive query. Or sent an email to the wrong customer. Or exposed PII in a log. CTO mandates "we need controls before next time."

Scenario 3 — Read the news: Saw the Vercel breach (April 19, 2026 — OAuth tokens stolen via MCP-related Context AI tool, $2M of data sold). Saw the Asana MCP breach (June 2025). Saw CVE-2025-6514 (437,000+ developer environments compromised). CTO budgets for MCP security before something happens.

What your product is NOT

- NOT an AI agent itself
- NOT an LLM gateway (that's Portkey, Helicone)
- NOT an integration platform (that's Composio)
- NOT a compliance report generator
- NOT an open-source self-hosted thing customers run themselves
- NOT a monitoring tool you bolt onto existing infrastructure

What your product IS

- A managed proxy/gateway service customers point their MCP traffic through
- A dashboard to see, search, and export every tool call
- A policy engine that blocks dangerous actions before they execute
- An OAuth 2.1 layer that makes MCP servers production-safe
- A free CLI scanner that shows shadow MCP servers in their environment

---

PART 2: TECHNICAL FOUNDATION — MCP DEEP DIVE (FACTS YOU NEED)

How MCP actually works (verified from modelcontextprotocol.io)

MCP has three actors:

Host — The application the user interacts with directly. Claude Desktop, Cursor, VS Code, Continue, custom apps. The host manages connections, enforces security policies, and coordinates between the AI model and MCP clients.

Client — A protocol client that maintains a 1:1 connection with a single MCP server. The host creates one client per server.

Server — Your MCP server. A lightweight process that responds to JSON-RPC requests. Exposes three primitives:

- **Tools** — Actions the AI can perform (e.g., `query_database`, `send_email`)
- **Resources** — Data the AI can read (e.g., file contents, API responses)
- **Prompts** — Reusable templates

How they communicate

Every message is **JSON-RPC 2.0**. Same format across all transports.

Request format:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": {
    "name": "get_company_info",
    "arguments": {"company_name": "Anthropic"}
  }
}
```

Response format:

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "content": [{"type": "text", "text": "..."}]
  }
```

```
}
```

Two transports — both matter for your product

Transport 1: stdio (Standard I/O)

- Local servers
- Host launches server as a subprocess
- Communication via stdin/stdout pipes
- Zero network setup required
- Used by 90% of current MCP servers
- Your Day 1 proxy works on this transport

Transport 2: Streamable HTTP (modern) / SSE (deprecated)

- Remote servers
- HTTP POST + Server-Sent Events for streaming
- Production-grade for cloud-deployed servers
- Supports OAuth 2.1 for auth
- The future of MCP — all serious deployments will move here

Critical fact: stdio servers don't need OAuth — the host process controls access. HTTP servers MUST have OAuth 2.1 with PKCE per the MCP spec.

The MCP message types your gateway must handle

You need to intercept and understand these JSON-RPC methods:

Method	What it means	Log it?	Block-able?
`initialize`	Client connecting to server	Yes (session start)	No
`tools/list`	What tools does this server have?	Yes	Rare
`tools/call`	Execute a specific tool	YES (most important)	YES
`resources/list`	What resources are available?	Yes	Rare

`resources/read`	Read specific data	YES	YES
`prompts/list`	Available prompts	Yes	No
`prompts/get`	Get specific prompt	Yes	No
`notifications/*`	Server-side events	Yes	No
`logging/setLevel`	Log level configuration	Yes	No

The 80/20: **`tools/call` and `resources/read` are 95% of what matters.** Get those right, the rest is housekeeping.

What a tool call actually contains (for your audit log)

```
{
  "jsonrpc": "2.0",
  "id": 42,
  "method": "tools/call",
  "params": {
    "name": "execute_sql",
    "arguments": {
      "query": "SELECT * FROM customers WHERE email='alice@example.com'"
    },
    "_meta": {
      "progressToken": "abc123"
    }
  }
}
```

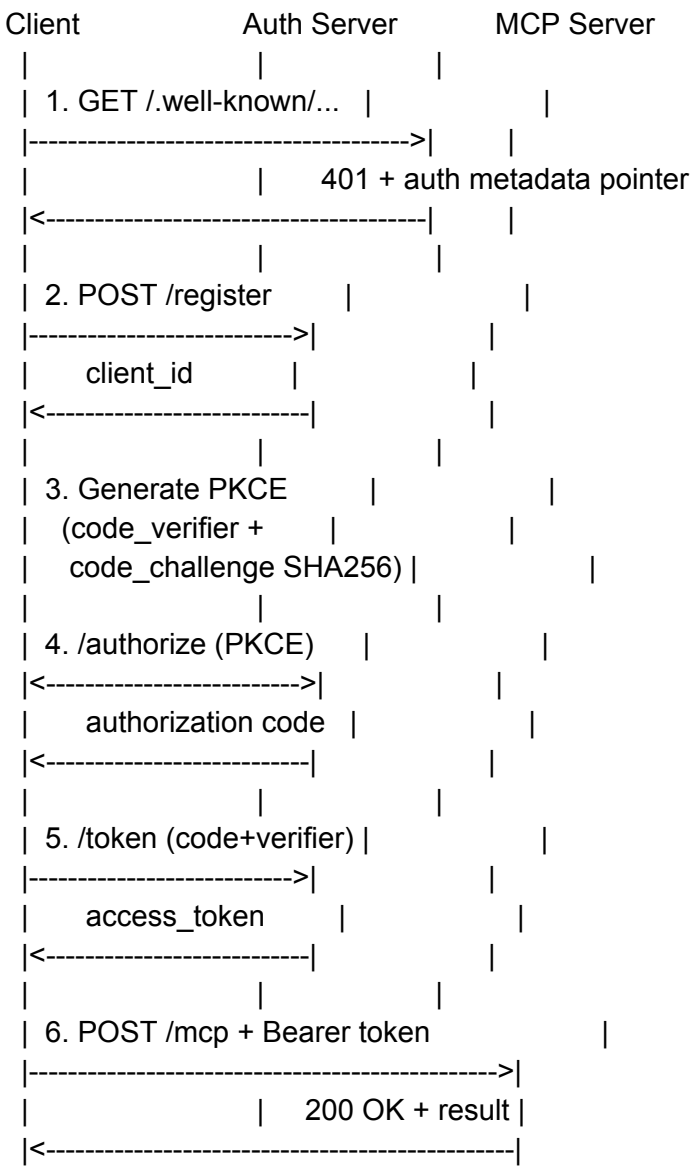
What you log:

- `agent_id` (which AI client)
- `tool_name` ("execute_sql")
- `parameters` (the full arguments object)
- `request_id` (42)
- `timestamp` (when received)
- `response` (full response from server)
- `duration_ms` (how long it took)

- `status` ("allowed" / "blocked" / "flagged")
- `policy_violations` (if any rules triggered)
- `pii_detected` (boolean + types found)

The OAuth 2.1 + PKCE flow (for HTTP MCP servers)

This is the production auth standard. You will need to support this. Verified flow from MCP spec:



The 4 RFCs you must implement:

- RFC 9728 — Protected Resource Metadata
- RFC 8707 — Resource Indicators (binds tokens to specific MCP server)

- RFC 8414 — Authorization Server Metadata
- RFC 7591 — Dynamic Client Registration

Critical security requirements (from MCP spec, NON-NEGOTIABLE):

- PKCE with SHA-256 (plain method NOT allowed under OAuth 2.1)
- HTTPS for all authorization endpoints
- Redirect URIs must be localhost OR HTTPS only
- Token audience validation (token only valid for the specific MCP server)
- Token isolation (don't pass tokens to other servers)
- Bearer token in `Authorization` header (NEVER in URL)

The CVEs you need to know

These are real, current vulnerabilities. You'll be asked about them by sophisticated buyers.

CVE-2025-6514 — `mcp-remote` npm package, RCE via OAuth flow injection. 437,000+ developer environments compromised. Caused by metadata fields not being sanitized. **Lesson:** Sanitize EVERY field from metadata discovery.

CVE-2025-49596 — Unauthenticated command injection. CVSS 9.4. **Lesson:** Validate all parameters before forwarding.

CVE-2025-53110 — Filesystem MCP sandbox escape. **Lesson:** Path traversal protection on file MCP servers.

CVE-2025-53109 — Symlink bypass on filesystem MCP. **Lesson:** Resolve symlinks before validation.

PART 3: THE COMPLETE PRODUCT — EVERY FEATURE

Layer 1: Free CLI Scanner (Acquisition Hook)

What it is

A single npm command (`npx mcp-security-scan`) that scans a developer's machine and outputs a security report on every MCP server they have configured.

What it scans

1. Claude Desktop config (`~/Library/Application Support/Claude/` on Mac, `%APPDATA%\Claude\` on Windows)
2. Cursor config (`~/.cursor/mcp.json`)
3. VS Code config (`~/.vscode/`, workspace settings)
4. Continue.dev config
5. Cline config
6. Windsurf config
7. Currently running processes (find any node/python processes that look like MCP servers)
8. npm/pip globally installed packages matching known MCP package patterns

What it checks for each server

1. **Auth status** — Does it use OAuth or static API keys? (most use static keys = risk)
2. **Hardcoded secrets** — Scan config args/env for patterns matching API keys, AWS credentials, database passwords
3. **Tool inventory** — Connect to the server and call `tools/list` — show user every tool exposed
4. **Permission scope** — For known servers, show what they can access (filesystem, GitHub, DBs, etc.)
5. **Server reputation** — Compare package name/source to known malicious patterns
6. **CVE matches** — Check installed versions against known vulnerabilities

Sample output

🔍 MCP Security Scan Report
Generated: 2026-05-01 14:23:01

Found 12 MCP servers configured.
Detected: 8 high-risk, 3 medium-risk, 1 low-risk.

———

🔴 HIGH RISK

1. server-postgres (Cursor)
   ⚠️ Hardcoded database credentials in config
   ⚠️ Has SELECT/INSERT/UPDATE/DELETE on all tables
   ⚠️ No audit log of queries
   Path: ~/.cursor/mcp.json

2. github-mcp-server (Claude Desktop)
   ⚠️ Personal access token in env var (no rotation)

⚠️ Token has admin scope
⚠️ CVE-2025-6514 vulnerability (mcp-remote 1.2.0)
Path: ~/Library/Application Support/Claude/

[8 more servers...]
———————————————————————————————————————

📊 Summary
Servers with hardcoded credentials: 9 (75%)
Servers without OAuth: 11 (92%)
Servers without audit logging: 12 (100%)

🛡️ Want to monitor and control these in production?
Try the managed gateway: https://[your-domain]/start

Build effort

3-5 days. Pure JavaScript/TypeScript. No backend needed.

Distribution

- Open source on GitHub
- Submit to Hacker News (Show HN format)
- Tweet thread with screenshots
- LinkedIn post showing real findings
- DEV.to article walking through the code
- Eventually: integrate with `npm audit`, GitHub security scanning

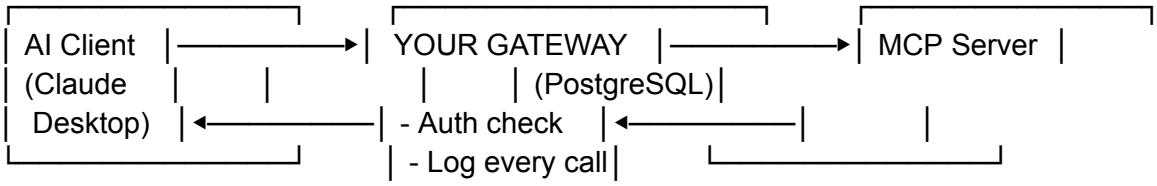
The conversion path

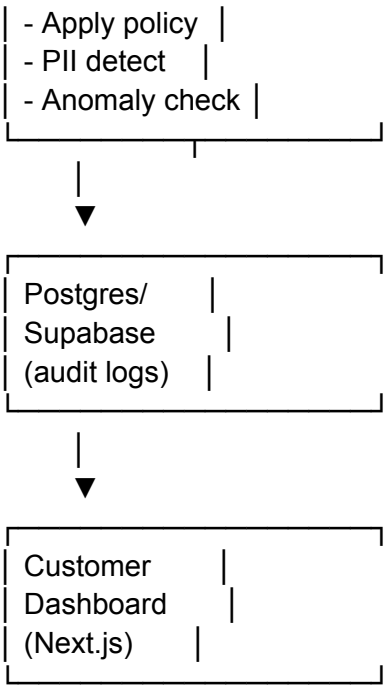
Free scan → User sees they have problems → CTA in output → Sign up for managed gateway

---

Layer 2: The Gateway Proxy (Core Product)

Architecture overview





Two gateway modes (you build both, customers pick)

Mode A: Local proxy (stdio) What you already built today. Customer adds a line to their `mcp.json`:

```
{
  "mcpServers": {
    "their-server": {
      "command": "node",
      "args": [
        "/path/to/your-gateway",
        "--api-key", "their-key",
        "--upstream-command", "node",
        "--upstream-args", "their-actual-server.js"
      ]
    }
  }
}
```

Your gateway spawns the real server as a subprocess, intercepts everything between Claude Desktop and the server.

Time to build: Already done. Just need to clean up.

Mode B: Remote HTTP gateway Customer points their AI client to your hosted endpoint:

```
{
  "mcpServers": {
    "their-server": {
      "url": "https://gateway.your-domain.com/their-server-id",
      "headers": {
        "Authorization": "Bearer their-access-token"
      }
    }
  }
}
```

You proxy to their actual MCP server (which they hosted somewhere), or to a managed instance you run for them.

Time to build: 7-10 days.

What the gateway does on every request

```
// pseudocode
async function handleToolCall(request) {
  const startTime = Date.now();

  // Step 1: Authenticate the caller
  const auth = await authenticateRequest(request);
  if (!auth.valid) return reject(401);

  // Step 2: Look up the customer's policies
  const policies = await getPoliciesForCustomer(auth.customerId);

  // Step 3: Pre-execution checks
  const piiDetection = await detectPII(request.params.arguments);
  const policyViolations = checkPolicies(request, policies);
  const anomalyScore = checkAnomalies(request, auth.customerId);

  // Step 4: Block if policy says so
  if (policyViolations.length > 0 && policies.blockOnViolation) {
    await logBlockedRequest(request, policyViolations);
    return rejectRequest("Policy violation: " + policyViolations[0].name);
  }

  // Step 5: Forward to upstream MCP server
  const response = await forwardToMCPServer(request);
```

```
  // Step 6: Post-execution checks (response also has PII)
  const responsePII = await detectPII(response);

  // Step 7: Log everything
  await logToolCall({
    customerId: auth.customerId,
    agentId: auth.agentId,
    toolName: request.params.name,
    parameters: request.params.arguments,
    response: response,
    durationMs: Date.now() - startTime,
    piiDetected: [...piiDetection, ...responsePII],
    policyViolations: policyViolations,
    anomalyScore: anomalyScore,
    status: policyViolations.length > 0 ? "blocked" : "allowed",
    timestamp: new Date()
  });

  // Step 8: Return response (possibly redacted)
  return policies.redactPII ? redactPII(response, responsePII) : response;
}
```

Database schema you need

```
-- Your existing table, expanded
CREATE TABLE tool_calls (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  created_at TIMESTAMPTZ DEFAULT NOW(),

  -- Tenant + agent
  customer_id UUID NOT NULL REFERENCES customers(id),
  agent_id TEXT NOT NULL,
  session_id UUID,

  -- The call itself
  server_name TEXT NOT NULL,
  tool_name TEXT NOT NULL,
  parameters JSONB,
  request_id TEXT,

  -- Response
  response JSONB,
  duration_ms INTEGER,
```

```
  -- Security signals
  status TEXT CHECK (status IN ('allowed', 'blocked', 'flagged', 'error')),
  policy_violations JSONB,
  pii_detected JSONB,
  anomaly_score FLOAT,

  -- Source tracking
  source_ip TEXT,
  user_agent TEXT
);

CREATE INDEX idx_tool_calls_customer ON tool_calls(customer_id, created_at DESC);
CREATE INDEX idx_tool_calls_status ON tool_calls(customer_id, status);
CREATE INDEX idx_tool_calls_tool ON tool_calls(customer_id, tool_name);

-- Customers
CREATE TABLE customers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email TEXT UNIQUE NOT NULL,
  company_name TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW(),

  api_key TEXT UNIQUE NOT NULL,
  plan TEXT DEFAULT 'free',

  -- Stripe
  stripe_customer_id TEXT,
  stripe_subscription_id TEXT,
  trial_ends_at TIMESTAMPTZ
);

-- Policies (rules)
CREATE TABLE policies (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  customer_id UUID NOT NULL REFERENCES customers(id),
  created_at TIMESTAMPTZ DEFAULT NOW(),

  name TEXT NOT NULL,
  description TEXT,
  enabled BOOLEAN DEFAULT TRUE,

  -- Rule logic stored as JSON
  rule_type TEXT, -- 'block_tool', 'block_param_pattern', 'rate_limit', etc.
```

```
  rule_config JSONB,

  action TEXT CHECK (action IN ('block', 'flag', 'alert')),
  alert_channels JSONB -- ['email', 'slack', etc.]
);

-- MCP servers being protected
CREATE TABLE mcp_servers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  customer_id UUID NOT NULL REFERENCES customers(id),
  name TEXT NOT NULL,
  transport TEXT CHECK (transport IN ('stdio', 'http')),
  upstream_config JSONB, -- command/args for stdio, URL for http
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Sessions (track an AI client's session)
CREATE TABLE sessions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  customer_id UUID NOT NULL REFERENCES customers(id),
  agent_id TEXT NOT NULL,
  started_at TIMESTAMPTZ DEFAULT NOW(),
  ended_at TIMESTAMPTZ,
  total_calls INTEGER DEFAULT 0
);
```

Layer 3: The Dashboard (How Customers See Value)

Pages to build

1. Live Activity Feed (`/dashboard`) The home page. Real-time stream of tool calls.

Top section — Summary cards:

- Total tool calls (last 24h)
- Blocked calls
- PII detections
- Active agents
- MCP servers monitored

Main section — Live table:

Time | Agent | Server | Tool | Status | Actions
14:23:01 | claude-prod | postgres | execute_sql | Allowed | View
14:23:04 | claude-prod | gmail | send_email | Blocked | View (PII)
14:22:58 | cursor-dev | filesys | read_file | Allowed | View
14:21:33 | claude-prod | postgres | delete_record | Flagged | Approve

Each row clicks into a detail drawer showing full request/response, parameters, PII detected, etc.

2. Tool Calls Browser (`/dashboard/calls`) Searchable, filterable history. SQL-style filters.

Filters:

- Date range
- Agent ID
- Tool name
- Status
- Has PII
- MCP server

Each call has detail page with:

- Full request payload
- Full response payload
- PII matches highlighted
- Policy evaluation result
- Anomaly score breakdown

3. Compliance Reports (`/dashboard/reports`) The killer feature for closing enterprise deals.

One-click PDF export with:

- Cover page (customer logo, date range)
- Executive summary (total calls, blocked, PII redacted)
- Authentication summary (OAuth status, token rotation)
- Detailed audit trail (paginated table of all calls)
- Policy compliance summary
- Encryption details (TLS 1.3 in transit, AES-256 at rest)
- Retention policy

This is what customers send to enterprise security teams. THIS feature alone justifies $299/month.

4. Policies (`/dashboard/policies`) List of active rules. Toggle on/off. Add new rules.

Pre-built rule templates:

- "Block any tool call containing credit card numbers"
- "Block any tool call between 10pm-8am UTC"
- "Rate limit any tool to 100 calls/hour"
- "Flag any DELETE operations"
- "Block tool calls from agents not in allowlist"
- "Require approval for tool X"

Each rule:

- Type
- Conditions (configurable)
- Action (block, flag, alert)
- Alert channels (email, Slack)

5. MCP Servers (`/dashboard/servers`) List of MCP servers being protected. Add new ones.

Per server:

- Name
- Transport (stdio/HTTP)
- Connection status
- Tool calls today
- Health check status
- Tool inventory (what tools it exposes)

6. Agents (`/dashboard/agents`) List of AI clients/agents calling through your gateway.

Per agent:

- ID (e.g., "claude-desktop", "cursor-vscode")
- First seen
- Last active
- Total calls
- Blocked rate
- Tools used most

7. Settings (`/dashboard/settings`)

- API keys (rotate, revoke)
- Team members (invite, RBAC — later)
- Billing (Stripe customer portal)
- Integrations (Slack, email alerts, webhooks)
- Data retention

Tech stack for dashboard

- **Frontend:** Next.js 15 (you know this) + Tailwind + shadcn/ui
- **Auth:** Clerk or Supabase Auth (don't build your own)
- **Realtime updates:** Supabase Realtime subscriptions
- **PDF generation:** `@react-pdf/renderer` or `pdfkit`
- **Charts:** Recharts (built into shadcn)
- **Tables:** TanStack Table

Build effort

10-14 days for v1 dashboard. Not 30. You can ship it.

Layer 4: Policy Engine

What it is

A rules engine that evaluates every tool call against configured policies before execution. Blocks, flags, or alerts.

Rule types to support (in priority order)

Tier 1 — Build first (Day 1 of policy engine):

1. **Block tool by name** — Block `delete_record` always
2. **Block tool by pattern** — Block any tool starting with `admin_`
3. **Allow only listed tools** — Whitelist mode
4. **Rate limit per tool** — Max 100 calls/hour for `send_email`
5. **Block by parameter pattern** — Block calls where SQL contains `DROP`

Tier 2 — Build after first customers:

6. **Time-of-day restrictions** — No destructive ops between 10pm-8am
7. **Require approval** — Some calls require human Slack approval
8. **Block by parameter PII** — Block if any parameter contains SSN
9. **Block external destinations** — Email tool can't send to non-corporate domains
10. **Require comment** — Calls with sensitive operations need a justification field

Tier 3 — Premium features:

11. **Anomaly-based blocking** — Block if anomaly score > threshold
12. **Cross-tool patterns** — Detect when agent reads sensitive data then calls external API

13. **Custom JavaScript rules** — Customer writes their own evaluator

Implementation approach

Don't over-engineer. Use simple JSON rule definitions:

```
type Rule = {
  id: string;
  name: string;
  type: 'block_tool' | 'block_param_regex' | 'rate_limit' | 'block_time_range';
  enabled: boolean;

  // Type-specific config
  config: {
    // For block_tool
    tool_names?: string[];

    // For block_param_regex
    param_path?: string; // e.g., "arguments.query"
    pattern?: string; // regex

    // For rate_limit
    tool_name?: string;
    max_calls?: number;
    window_seconds?: number;

    // For block_time_range
    start_hour?: number;
    end_hour?: number;
    timezone?: string;
  };

  action: 'block' | 'flag' | 'alert';
  alert_channels?: ('email' | 'slack' | 'webhook')[];
};
```

Evaluator runs all enabled rules for the customer in sequence. First matching rule with `block` action stops execution.

Build effort

5-7 days for Tier 1.

Layer 5: PII Detection

What it does

Scans every parameter and response for sensitive data. Flags or redacts.

What it detects (in priority order)

Pattern-based (regex, fast):

1. Credit card numbers (Luhn check)
2. SSN (US)
3. Phone numbers
4. Email addresses (sometimes PII, sometimes not)
5. IP addresses (sometimes)
6. AWS keys (`AKIA...`)
7. Stripe keys (`sk_live_...`, `pk_live_...`)
8. GitHub tokens (`ghp_...`, `gho_...`)
9. Slack tokens (`xoxb-...`, `xoxp-...`)

LLM-based (Claude API call, slower but smarter): 10. Names (when in customer context) 11. Addresses 12. Medical information 13. Financial account numbers 14. Custom PII types per customer

Implementation

Phase 1 — Regex only (fast, ship first):

Use the `presidio-analyzer` library or build your own regex set. Sub-millisecond detection.

```
const PII_PATTERNS = {
  credit_card: /\b(?:4\d{3}|5[1-5]\d{2}|6011|3[47]\d{2})[-\s]?\d{4}[-\s]?\d{4}[-\s]?\d{4}\b/g,
  ssn: /\b\d{3}-\d{2}-\d{4}\b/g,
  aws_key: /\bAKIA[0-9A-Z]{16}\b/g,
  stripe_secret: /\bsk_live_[0-9a-zA-Z]{24,}\b/g,
  github_token: /\bghp_[0-9a-zA-Z]{36}\b/g,
  email: /\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b/g,
};

function detectPII(text: string): PIIMatch[] {
  const matches = [];
  for (const [type, pattern] of Object.entries(PII_PATTERNS)) {
    const found = text.match(pattern) || [];
    for (const match of found) {
```

```
      // Validate (e.g., Luhn check for credit cards)
      if (validate(type, match)) {
        matches.push({ type, value: match, position: text.indexOf(match) });
      }
    }
  }
  return matches;
}
```

Phase 2 — LLM augmentation (premium tier):

Send tool call to Claude API with a prompt asking it to identify PII. Cache results aggressively.

Redaction modes

- **Detect only** — Log the PII, don't modify request
- **Mask** — Replace `4111-1111-1111-1234` with `4111-XXXX-XXXX-1234`
- **Hash** — Replace with SHA-256 hash (preserves uniqueness for analytics)
- **Block** — Reject the entire request

Build effort

3-5 days for regex-based. Add LLM later.

Layer 6: OAuth 2.1 Authentication

When you need this

Not on Day 1. stdio MCP servers don't need OAuth. Your first 5-10 customers will probably be on stdio.

You need it when:

- A customer wants to put your gateway in front of HTTP MCP servers
- A customer wants different agents to have different permissions
- An enterprise prospect asks "how do you authenticate agents?"

What you build

You become an OAuth 2.1 authorization server. Implement these endpoints:

POST /.well-known/oauth-authorization-server
  Returns metadata document (RFC 8414)

POST /register
  Dynamic client registration (RFC 7591)

GET /authorize
  PKCE-protected authorization flow

POST /token
  Token issuance with PKCE verification

POST /introspect
  Token introspection (RFC 7662)

POST /revoke
  Token revocation

Don't build this yourself

Use a managed OAuth provider. Verified options that work for MCP in 2026:

- **Scalekit** — Drop-in OAuth specifically for MCP. Recommended.
- **Auth0** — Enterprise-grade, expensive
- **Clerk** — Newer, simpler, good for B2B
- **Supabase Auth** — Free tier, you already use Supabase
- **Keycloak** — Self-hosted, free, complex

My recommendation: Use Supabase Auth for your DASHBOARD authentication. Use Scalekit (or implement minimal OAuth flow with `mcp` SDK helpers) for MCP-level OAuth.

The flow customers go through

1. Customer adds an HTTP MCP server to your gateway
2. Customer's AI client connects to your gateway URL
3. Gateway responds 401 with metadata pointer
4. AI client discovers OAuth endpoints via `.well-known`
5. AI client registers as OAuth client (dynamic registration)
6. AI client does PKCE auth flow
7. Gets access token bound to your gateway URL
8. Sends every request with `Authorization: Bearer <token>`
9. Gateway validates token, identifies which agent/user
10. Gateway logs and forwards to upstream MCP server

Build effort

If you use Scalekit or similar: 5-7 days. If you build from scratch: 3-4 weeks.

Recommendation: Don't build from scratch. Use Scalekit. Their entire pitch is "drop in OAuth for MCP in minutes."

Layer 7: Anomaly Detection

What it does

Looks at patterns of tool calls and flags unusual behavior. NOT for v1. Build after first customers.

Approaches (cheap to expensive)

Approach 1: Statistical baselines (cheap, ship first) Per agent, track:

- Average calls per hour
- Most common tools
- Average parameter sizes
- Common destinations

Flag if current behavior deviates >3 standard deviations.

Approach 2: Sequence patterns (medium) Track common tool call sequences. Flag novel sequences.

Approach 3: Claude API analysis (expensive but smartest) Send recent tool call history to Claude API with prompt:

> "Below is the recent tool call history for an AI agent. Is anything unusual or potentially malicious? Score 0-100."

Build effort

10-14 days for Approach 1. Don't ship until you have customers asking.

Layer 8: Billing & Self-Serve Signup

Stripe integration

Use Stripe Checkout (not custom payment forms). Subscription tiers:

```
const TIERS = {
  free: {
    name: 'Free',
    price: 0,
    limits: {
      tool_calls_per_day: 100,
      log_retention_days: 7,
      mcp_servers: 1,
      policies: 0,
      pii_detection: false,
      pdf_export: false,
    }
  },
  starter: {
    name: 'Starter',
    price: 299,
    stripe_price_id: 'price_xxx',
    limits: {
      tool_calls_per_day: 10000,
      log_retention_days: 30,
      mcp_servers: 3,
      policies: 5,
      pii_detection: true,
      pdf_export: true,
    }
  },
  growth: {
    name: 'Growth',
    price: 799,
    stripe_price_id: 'price_yyy',
    limits: {
      tool_calls_per_day: 100000,
      log_retention_days: 90,
      mcp_servers: 10,
      policies: 25,
      pii_detection: true,
      pdf_export: true,
      slack_alerts: true,
      anomaly_detection: 'basic',
    }
```

```
  },
  scale: {
    name: 'Scale',
    price: 2000,
    stripe_price_id: 'price_zzz',
    limits: {
      tool_calls_per_day: -1, // unlimited
      log_retention_days: 365,
      mcp_servers: -1,
      policies: -1,
      pii_detection: true,
      pdf_export: true,
      slack_alerts: true,
      anomaly_detection: 'advanced',
      api_access: true,
      priority_support: true,
    }
  }
};
```

Signup flow

1. Land on pricing page
2. Click "Start free trial" on any paid tier
3. Email + password signup (Supabase Auth)
4. Stripe Checkout for billing details (14-day free trial, no charge)
5. Generate API key
6. Show "Get started" page with one-line install command
7. Run scanner first (warm-up)
8. Set up first MCP server
9. Show first tool calls flowing through dashboard

The "aha moment" target: Signup → first tool call logged in dashboard in under 10 minutes.

Build effort

3-5 days for Stripe + signup flow.

PART 4: WHAT YOU NEED — COMPLETE DEPENDENCY LIST

Services & Subscriptions (start of build)

Service	Purpose	Cost
Vercel	Frontend hosting	Free → $20/mo
Supabase	Database + Auth + Storage	Free → $25/mo
Stripe	Payments	2.9% + 30¢ per txn
Resend or Postmark	Transactional email	Free → $15/mo
Sentry	Error tracking	Free → $26/mo
GitHub	Code hosting	Free
Domain	Your branded domain	$12/year
Cloudflare	DNS + CDN	Free

Monthly burn at start: ~$50-100/month. Sustainable.

Optional but recommended

Service	Purpose	Cost
Apollo.io	Lead enrichment for outbound	$99/mo
Instantly.ai	Cold email automation	$37/mo
Loom	Demo video recording	Free → $15/mo
Cal.com	Booking demo calls	Free
PostHog	Product analytics	Free → $0/mo at low volume
Clerk or Scalekit	OAuth for MCP	Free → $25/mo

Tech stack (no debate, just pick these)

Frontend:

- Next.js 15 (App Router)
- TypeScript
- Tailwind CSS
- shadcn/ui (component library)
- TanStack Query (data fetching)
- TanStack Table (data tables)
- Recharts (charts)
- Zustand or React Context (state)

Backend:

- Node.js 20+
- Hono or Express (API framework)
- TypeScript
- Supabase JS SDK
- @modelcontextprotocol/sdk (official MCP SDK)
- Zod (schema validation)

Database:

- PostgreSQL via Supabase
- Use `pgcrypto` for token hashing
- Row-level security (RLS) for multi-tenancy

Infrastructure:

- Vercel for Next.js + serverless functions
- Vercel Postgres or Supabase Postgres
- Cloudflare in front of everything (DDoS protection, WAF)

Why not other options?

- Don't use SvelteKit/Remix — smaller community, fewer libs
- Don't use AWS Lambda directly — Vercel is easier and same underneath
- Don't self-host PostgreSQL — Supabase is fine, free tier is generous
- Don't use MongoDB — JSON queries on Postgres are fine

Skills you need

You said you know:

- Next.js ✅
- TypeScript ✅

- Python ✅ (won't need much for this)
- Claude API ✅

Skills you'll need to learn (in priority order):

1. **Postgres advanced features** — Indexes, JSON queries, RLS. (1-2 days of learning)
2. **OAuth 2.1 implementation** — Even if using Scalekit, understand the flow. (3-5 days reading)
3. **Streaming HTTP / SSE** — For remote MCP. (2-3 days)
4. **Stripe Checkout + webhooks** — Subscription handling. (2 days)

What you don't need

- Don't need React Native (no mobile app)
- Don't need GraphQL (REST is fine)
- Don't need Kubernetes (Vercel handles deployment)
- Don't need a separate auth service (Supabase Auth is fine for users)
- Don't need Redis caching (Postgres is fast enough at this scale)
- Don't need monitoring beyond Sentry
- Don't need TimescaleDB (regular Postgres + indexes are fine until 100M+ rows)

PART 5: THE 30-DAY BUILD PLAN

Week 1 (Days 1-7): Solidify Foundation

You have: Working stdio gateway logging to Supabase. ✅

This week:

Day 1 (today/tomorrow):

- Clean up `proxy.mjs` — proper error handling, response logging, duration tracking
- Add response logging to schema and code
- Test with multiple MCP servers (not just Sevlex)

Day 2-3:

- Set up new Next.js project for dashboard
- Implement Supabase Auth (signup, login, magic links)
- Basic protected routes structure

Day 4-5:

- Live activity feed page (`/dashboard`)
- Real-time updates via Supabase Realtime
- Tool call detail drawer

Day 6-7:

- Tool calls history page (`/dashboard/calls`)
- Filters (date, agent, tool, status)
- Search functionality

Goal at end of Week 1: Customer can sign up, get API key, install gateway, see real-time activity in dashboard.

Week 2 (Days 8-14): Free Scanner + Public Launch Prep

Day 8-10:

- Build CLI scanner (`mcp-security-scan` npm package)
- Scans Claude/Cursor/VS Code configs
- Detects hardcoded secrets, OAuth status, tool inventory
- Open source on GitHub

Day 11:

- PDF compliance export feature
- Use `@react-pdf/renderer`
- Customer logo, branding, executive summary

Day 12-13:

- Public landing page
- Pricing page (4 tiers, public)
- Comparison page (vs Runlayer, vs MintMCP)
- Documentation site (Mintlify or Nextra)

Day 14:

- Hacker News submission day (Tuesday or Wednesday 8am PT)
- Title: "Show HN: Free CLI to find shadow MCP servers in your environment"
- Be present in comments for 4 hours

Goal at end of Week 2: First 100 GitHub stars on scanner. First 10 trial signups from HN.

Week 3 (Days 15-21): Policy Engine + Stripe

Day 15-17:

- Policy engine implementation
- Tier 1 rules: block_tool, block_param_regex, rate_limit, block_time_range
- Policies UI in dashboard
- Pre-built rule templates

Day 18-19:

- Stripe integration
- Checkout flow
- Subscription webhook handling
- Plan limit enforcement
- Customer portal (Stripe-hosted)

Day 20-21:

- PII detection (regex-based)
- Show PII matches in dashboard
- PII redaction option in policies

Goal at end of Week 3: Product is fully self-serve. Customer can sign up, pay, configure policies, see PII detection working.

Week 4 (Days 22-30): Polish + First Customer

Day 22-23:

- Slack alert integration
- Email alert integration
- Webhook support

Day 24-25:

- Onboarding flow (interactive setup wizard)
- Email sequence (Day 0, 1, 3, 7, 12, 14)
- "Aha moment" optimization

Day 26-27:

- Documentation: getting started, OAuth setup, policy examples
- Demo video (Loom, 3 minutes)
- Comparison content (Runlayer alternative, MintMCP alternative)

Day 28-30:

- Outbound to ICP list (50 messages/day)
- First demo calls
- Close first customer

Goal at end of Day 30: First paying customer. $299 MRR.

PART 6: WHAT TO BUILD AFTER 30 DAYS

Month 2 (Days 31-60)

Higher-priority features:

- OAuth 2.1 server (use Scalekit or implement minimal version)
- Remote HTTP MCP gateway support
- Anomaly detection (statistical approach)
- Team management / RBAC

Distribution:

- 200 outbound emails/week
- Daily LinkedIn content
- Hacker News Launch #2: "I scanned 1,000 MCP servers, here's what I found"
- First case study (from first customer)

Goal: 5 paying customers. $2,500-5,000 MRR.

Month 3 (Days 61-90)

Features:

- Advanced policy rules (time-based, cross-tool patterns)
- Custom PII patterns per customer
- API access for paid tiers
- Audit log search via API

Distribution:

- Hacker News Launch #3: full product launch
- "Self-serve alternative to Runlayer"

- Multiple comparison posts ranking on Google
- YC Fall 2026 application submitted

Goal: 10+ paying customers. $5,000-10,000 MRR. YC interview booked.

PART 7: WHAT TO BUILD AFTER YC

After YC funding (assume Series A in 2027):

Enterprise features:

- SSO / SAML
- Dedicated VPC deployment
- SOC 2 Type II certification (start at month 6, finishes ~month 18)
- HIPAA compliance
- Custom data retention policies
- Dedicated Slack channel support
- SLA guarantees (99.95%+)

Advanced features:

- ML-based threat detection (like Runlayer's ToolGuard)
- MCP server registry/catalog
- Tool call sandboxing
- Multi-region deployment

Don't build any of this before YC. Series A buyers don't need it. Solo founders can't build it. It's a trap.

PART 8: COMMON MISTAKES TO AVOID

Technical mistakes

1. Logging tokens in audit trail Never. Log tool name, parameters (with PII redacted), and response shape. Don't log raw OAuth tokens — if your DB is breached, attacker has all customer credentials.

2. Putting tokens in URLs Per OAuth 2.1 spec, NEVER. Always use `Authorization: Bearer` header.

3. Not validating token audience Each token must be bound to YOUR gateway URL via the `resource` parameter. If you accept tokens issued for any resource, attacker can use a stolen token from elsewhere.

4. Shipping without rate limiting Single misbehaving customer can DDoS your service. Implement per-customer rate limits from Day 1.

5. Not sanitizing metadata fields CVE-2025-6514 happened because of unsanitized metadata fields. Sanitize EVERYTHING from external sources.

6. Storing customer data without RLS Use Supabase Row-Level Security from the start. One bug, customers see each other's data, you're done.

7. No backup strategy Audit logs are valuable. Backup daily. Test restore quarterly.

8. Building OAuth 2.1 from scratch Use Scalekit. The MCP spec compliance is hard. Scalekit specifically builds for MCP.

Product mistakes

9. Adding too many features before customers Ship what's in this doc. Stop. Get customers. Let them tell you what's missing.

10. Free tier that's too generous Give scanner free. Give 100 tool calls/day free. Don't give 10,000 free — paying tier shouldn't be 100x more useful.

11. Pricing too low $299 is right. Don't price at $49 because "you're new." Series A companies can pay $299. They can't pay $49 — feels like a hobby project.

12. Selling to wrong ICP Don't sell to:

- Pre-seed startups (no budget)
- F500 enterprises (too slow, need SOC 2)
- Individual developers (use free tier forever)

DO sell to:

- Series A AI-native companies, 50-300 employees, CTO buying

13. Trying to win on features vs Runlayer You can't. Win on access (self-serve, public pricing, 30-min deploy).

14. Hiring too early Don't hire anyone until $20k+ MRR. Then hire one person — engineer, not salesperson.

Distribution mistakes

15. Cold messaging without content first Per 2026 LinkedIn data: cold = 1.7% conversion. Content + warm = 14.6%. Build content presence before outbound.

16. Hiding pricing Every competitor does this. Don't. Public pricing is your differentiator.

17. Long sales calls You don't have time. Self-serve onboarding. Async demos via Loom.

18. Not measuring Track: signups/week, free→paid conversion, MRR, churn. Weekly. In a spreadsheet.

PART 9: ONE FINAL CHECKLIST

Before you start tomorrow, verify you have:

Accounts:

- [ ] Vercel account
- [ ] Supabase account (you already have this)
- [ ] Stripe account (start in test mode)
- [ ] GitHub account
- [ ] Domain registered (e.g., trysevlex.com or get-mcpsec.com)
- [ ] Cloudflare account (point domain DNS here)
- [ ] Resend or Postmark for emails
- [ ] PostHog for analytics (optional but recommended)

Repositories:

- [ ] `mcp-security-gateway-proxy` (your gateway code)
- [ ] `mcp-security-gateway-dashboard` (Next.js app)
- [ ] `mcp-security-scan` (CLI scanner — public, open source)

Documents:

- [ ] Privacy policy (use Termly or RocketLawyer)
- [ ] Terms of service
- [ ] Acceptable use policy

- [ ] Security policy (one-page summary you can share)

Communication channels:

- [ ] Customer support email (`support@yourdomain.com`)
- [ ] Sales email (`hi@yourdomain.com`)
- [ ] Twitter handle
- [ ] LinkedIn profile updated
- [ ] Cal.com link for demo bookings

PART 10: THE NORTH STAR

You have a working product foundation today. Most founders don't.

You have proven outreach skills. Most founders don't.

You have 6 hours/day of focused work time. Most founders don't.

You have a real, validated market. The 30+ competitors prove it. The Vercel breach proves it. The Felicis blog post on Runlayer proves it.

You have a real, identifiable gap. Self-serve at $299 for Series A AI startups. Nobody is sitting in that seat.

What you don't have:

- A famous co-founder (don't need one for first 10 customers)
- $11M (don't need it — your tech costs are <$100/month)
- A US network (build it through content, not through introductions)
- SOC 2 (don't need it for Series A buyers — they don't ask)

The single number you track for the next 90 days: paying customers.

Not MRR. Not signups. Not GitHub stars. Not LinkedIn followers.

Paying customers.

10 of them by Day 90. Apply YC.

Everything else in this document supports that one number.

Build the dashboard tomorrow. Send the messages. Ship the scanner. Launch on Hacker News.

The product is real. The market is real. The path is clear.

Now go execute.

END OF DOCUMENT

This is your A-to-Z product playbook. Reference it when you're stuck. Update it as you learn. Don't deviate without good reason.

The first 10 customers will tell you 80% of what to build next. Until then, this document is the truth.